

Teraport Website – Handbuch

Anleitung: Projektstruktur · Redaktionelle Änderungen · Veröffentlichung · Menüstruktur

Stand: 12.07.2026 · Repository: github.com/RaphaSanto/tpwebsite · Technik: Astro 5 (statischer Site-Generator), Node ≥ 20

A · Struktur des Development-Projekts

A.1 Überblick

Die Website ist eine **statische Seite**: Aus Inhalten (Markdown-Dateien) und Vorlagen (Astro-Komponenten) erzeugt der Befehl `npm run build` fertiges HTML im Ordner `dist/`. Es gibt **keine Datenbank und kein CMS** – dadurch keine Angriffsfläche wie beim gehackten WordPress. Alles ist in Git versioniert; der Branch `main` ist der einzige, verbindliche Stand.

A.2 Ordnerstruktur

```
tpwebsite/
├── src/
│   ├── content/           INHALTE (Markdown) – hier findet Redaktion statt
│   │   ├── news/de|en/   Newsmeldungen (je Sprache eine Datei pro Beitrag)
│   │   ├── products/de|en/ Produktseiten: mido, veo, toolkit, connect
│   │   └── pages/de|en/   Seitentexte: Unternehmen, Services, Impressum, Datenschutz ...
│   ├── components/       Bausteine: Header, Footer, Hero, Karten ...
│   ├── layouts/          Seitengerüst (SiteLayout/BaseLayout)
│   ├── pages/            Routen: index, [product], news/..., kontakt, en/... (Spiegel)
│   ├── assets/
│   │   ├── brand/        Logo (teraport-logo.png, 480×120)
│   │   └── hero/         Hero-Grafik der Startseite (produkt-screenshot.png, 500×400)
│   ├── styles/global.css  Farb-/Design-Variablen (zentrale Stellschraube)
│   └── i18n/             Menü-Beschriftungen & Routen-Paare DE/EN
├── public/assets/uploads/ News-Bilder (werden 1:1 ausgeliefert)
├── scripts/              einmalige Migrations-Skripte (aus dem WordPress-Backup)
├── docs/                 Spezifikation, Pläne, dieses Handbuch, Brandbook
├── dist/                 BUILD-ERGEBNIS (generiert, nicht bearbeiten!)
├── news-export/, wordpress-restore/, *.sql.gz  Backup-Quellen (nur lokal, nicht in Git)
└── package.json          Befehle (npm run ...)
```

A.3 Die wichtigsten Befehle

Befehl	Zweck
<code>npm install</code>	Einmalig bzw. nach Updates: Abhängigkeiten installieren
<code>npm run dev</code>	Lokale Vorschau auf <code>http://localhost:4321</code> – aktualisiert sich live bei jeder Dateiänderung
<code>npm run build</code>	Produktions-Build nach <code>dist/</code> (prüft dabei alle Inhalte)
<code>npm run preview</code>	Den fertigen Build lokal ansehen (wie er live aussähe)
<code>npm test</code>	Automatische Tests (37 Stück) ausführen

⚠ Wichtig – `npm run migrate` nie im Alltag ausführen!

Die Skripte `migrate` / `import:news` / `extract:pages` dienen der einmaligen Übernahme aus dem WordPress-Backup. Sie **leeren und überschreiben** die Ordner `src/content/news/` und `src/content/pages/` – manuell erstellte Newsmeldungen und redaktionelle Korrekturen gingen dabei verloren. Nur nach Rücksprache verwenden.

B · Redaktionelle Änderungen umsetzen

B.0 Grundprinzip (gilt für alle Beispiele)

1. Datei bearbeiten (Editor: z. B. VS Code)
2. Vorschau prüfen: `npm run dev` → `http://localhost:4321`
3. Änderung sichern: `git add ... / git commit / git push` (siehe Abschnitt C)
4. Veröffentlichen (siehe Abschnitt C)

Alle Inhalte sind zweisprachig: **Jede Änderung in DE hat i. d. R. ein Gegenstück in EN** (gleicher Dateiname bzw. Spiegelordner `en/`).

B.1 Beispiel 1 – Text bearbeiten (Produktseite mido)

1. Datei öffnen: `src/content/products/de/mido.md`
2. Der Kopfbereich zwischen den `---`-Zeilen („Frontmatter“) steuert Titel/Teaser:

```
---
title: "mido"
lang: de
slug: "mido"
order: 1
teaser: "Werkzeugkostenkalkulation aus 3D-CAD-Modellen – schnell, transparent, nachvollziehbar."
---
```

3. Darunter steht der Seitentext als Markdown: `## Überschrift`, `- Aufzählung`, `**fett**`, `[Linktext]`
`(/kontakt/)`
4. Englischs Gegenstück nicht vergessen: `src/content/products/en/mido.md`

Gleiches Muster für `veo`, `toolkit`, `connect` sowie die Seiten unter `src/content/pages/...` (Unternehmen, Services, Impressum, Datenschutz).

B.2 Beispiel 2 – Bilder austauschen

Bild	Datei (einfach überschreiben)	Format
Logo im Header	<code>src/assets/brand/teraport-logo.png</code>	480 × 120 px, PNG transparent
Hero-Grafik Startseite	<code>src/assets/hero/produkt-screenshot.png</code>	500 × 400 px (5:4); andere Formate werden vom Passepartout-Rahmen automatisch beschnitten
News-Beitragsbild	<code>public/assets/uploads/JAHR/MONAT/name.jpg</code>	frei; Anzeige beschneidet auf 16:7

Nach dem Überschreiben genügt Speichern – die Vorschau (`npm run dev`) zeigt das neue Bild sofort.
Dateiname beibehalten = keine weiteren Änderungen nötig.

B.3 Beispiel 3 – Newsmeldung veröffentlichen

1. **Bild ablegen** (optional): z. B. `public/assets/uploads/2026/07/messe-xyz.jpg`
2. **Deutsche Meldung anlegen:** neue Datei `src/content/news/de/2026-07-15-teraport-auf-der-messe-xyz.md`
(Namensschema: `JJJJ-MM-TT-titel-in-kleinbuchstaben.md`)

```
---
title: "Teraport auf der Messe XYZ"
date: 2026-07-15
lang: de
slug: "teraport-auf-der-messe-xyz"
translation: "2026-07-15-teraport-at-xyz-fair.md"
image: "/assets/uploads/2026/07/messe-xyz.jpg"
---
```

Hier steht der Meldungstext. Absätze durch Leerzeilen trennen.
Markdown ist erlaubt: ****fett****, Links, Aufzählungen.

3. **Englische Meldung anlegen:** `src/content/news/en/2026-07-15-teraport-at-xyz-fair.md` – gleiches Schema mit `lang: en`, englischem `title` / `slug` / Text und `translation:` auf den deutschen Dateinamen. Über `translation` verknüpft der Sprachumschalter beide Meldungen.
4. **Prüfen:** `npm run dev` → Meldung erscheint automatisch unter `/news/` und (bei Aktualität) auf der Startseite; Detailseite und EN-Version anklicken.

Felder-Referenz: Pflicht: `title` , `date` , `lang` (de|en), `slug` . Optional: `translation` (Dateiname des Gegenstücks), `image` (Pfad unter `/assets/uploads/...`). Ohne `image` erscheint die Karte ohne Bild – kein Fehler.

C · Veröffentlichen (Deploy)

C.1 Schritt 1 – Änderungen prüfen und sichern (immer)

```
# im Projektordner c:\Users\rheilig\tpwebsite
npm test           # Tests müssen grün sein
npm run build      # Build muss fehlerfrei durchlaufen

git status         # zeigt geänderte Dateien
git add .          # Änderungen vormerken
git commit -m "News: Messe XYZ veröffentlicht"
git push           # nach GitHub sichern (main)
```

Damit ist der Stand versioniert und auf GitHub gesichert – **aber noch nicht öffentlich sichtbar**.

C.2 Schritt 2 – Website veröffentlichen

Aktueller Stand (07/2026): Ein Hosting-Ziel ist noch nicht eingerichtet – das ist der nächste Projektschritt („Plan 3“: Deployment, SEO, Kontaktformular). Bis dahin gilt das manuelle Verfahren C.2a; nach Einrichtung des Hostings das automatische Verfahren C.2b.

C.2a Manuell (Webpace/FTP)

1. `npm run build` ausführen → fertige Website liegt in `dist/`
2. Kompletten **Inhalt** von `dist/` in das Webroot des Hosters hochladen (FTP/SFTP oder Hosting-Panel), vorhandene Dateien ersetzen
3. Im Browser prüfen: Startseite, eine Produktseite, eine News, Sprachumschalter

C.2b Automatisch (Ziel-Setup, empfohlen)

Nach Anbindung an einen Dienst wie **Cloudflare Pages** oder **Netlify** entfällt C.2a vollständig: Jeder `git push` auf `main` baut und veröffentlicht die Seite automatisch (1–2 Minuten). Veröffentlichen = Schritt C.1, fertig.

C.3 Checkliste vor jeder Veröffentlichung

- ☐ DE- und EN-Version der Änderung vorhanden?
- ☐ `npm test` grün, `npm run build` fehlerfrei?
- ☐ Vorschau geprüft (Desktop + schmales Fenster/Mobil)?
- ☐ Keine Backup-Dateien versehentlich committet? (`git status` darf keine `.sql/.zip/wordpress-restore` zeigen – ist per `.gitignore` geschützt)
- ☐ Commit gepusht (`git push`)?

C.4 Hilfe & Wiederherstellung

- Letzte Änderung verwerfen (noch nicht committet): `git checkout -- pfad/zur/datei`

- Zu einem früheren Stand zurück: GitHub-Historie ansehen → `git revert <commit>` (erzeugt sauberen Gegen-Commit)
- Jeder Stand ist über Git vollständig wiederherstellbar – nichts geht verloren, solange committet und gepusht wurde

D · Zielstruktur der Website & Menü anpassen

D.1 Zielstruktur (Sitemap mit DE-↔-EN-Zuordnung)

```
Hauptnavigation
├─ Produkte ▾ (Dropdown)
│   ├── mido      /mido/      ↔ /en/mido/
│   ├── veo       /veo/       ↔ /en/veo/
│   ├── toolkit   /toolkit/   ↔ /en/toolkit/
│   └── connect   /connect/   ↔ /en/connect/
├─ Services       /services/  ↔ /en/services/
├─ Unternehmen    /unternehmen/ ↔ /en/company/
├─ News           /news/       ↔ /en/news/
│   └── Beitrag    /news/<slug>/ ↔ /en/news/<slug>/ (über translation-Feld verknüpft)
├─ Kontakt        /kontakt/    ↔ /en/contact/
└─ Sprachumschalter DE|EN

Startseite        /           ↔ /en/
Footer: Impressum /impressum/  ↔ /en/legal-notice/
      Datenschutz /datenschutz/ ↔ /en/data-privacy/
```

Wo steht die DE-↔-EN-Zuordnung? Für alle festen Seiten in genau einer Datei: `src/i18n/routes.ts` → Liste `routePairs` (je Zeile ein Paar, z. B. `{ de: '/unternehmen/', en: '/en/company/' }`). Der Sprachumschalter nutzt diese Tabelle. Nur **News-Beiträge** werden anders verknüpft: über das Feld `translation:` im Kopf der News-Datei, das den Dateinamen des Gegenstücks in der anderen Sprache nennt (siehe B.3).

D.2 Wo die Navigation definiert ist (4 Dateien)

Datei	Steuert
<code>src/components/Header.astro</code>	Aufbau & Reihenfolge des Menüs: welche Punkte, Dropdown-Inhalt, Links (DE- und EN-Ziel je Eintrag)
<code>src/i18n/ui.ts</code>	Beschriftungen in beiden Sprachen (z. B. <code>nav.company</code> = „Unternehmen“/ „Company“)
<code>src/i18n/routes.ts</code>	DE-↔-EN-Routenpaare für den Sprachumschalter
<code>src/components/Footer.astro</code>	Footer-Links (Impressum, Datenschutz) und Adresse

Die Seiten selbst liegen unter `src/pages/...` (DE) und `src/pages/en/...` (EN); Produktseiten entstehen automatisch aus `src/content/products/...`.

D.3 Rezepte zum Anpassen

R1 – Menüpunkt umbenennen

Nur die Beschriftung ändern → `src/i18n/ui.ts`, in beiden Sprachblöcken den Wert anpassen (z. B. `'nav.services': 'Leistungen'`). Fertig – Struktur bleibt unverändert.

R2 – Reihenfolge im Menü ändern

In `src/components/Header.astro` die `<a>`-Zeilen im `<nav>`-Block umsortieren. Reihenfolge der **Produkt-Kacheln** auf der Startseite: Feld `order:` im Kopf der Produktdateien (mido = 1, veo = 2 ...).

R3 – Produkt ins Dropdown/als Kachel hinzufügen (4 Schritte)

1. Inhalt anlegen: `src/content/products/de/neuprodukt.md` + `.../en/neuprodukt.md` (Frontmatter wie in B.1: title, lang, slug, order, teaser). → Seite `/neuprodukt/` und Startseiten-Kachel entstehen **automatisch**.
2. Sprachpaar eintragen: in `src/i18n/routes.ts` eine Zeile `{ de: '/neuprodukt/', en: '/en/neuprodukt/' },`
3. Dropdown-Link ergänzen: in `Header.astro` im `.menu`-Block `<a href={p('/neuprodukt/', '/en/neuprodukt/')}>neuprodukt</a>`
4. Prüfen: `npm test && npm run build`, Vorschau ansehen

R4 – Produkt/Kachel vorübergehend ausblenden

Nur die Startseiten-Kachel (Seite bleibt erreichbar): in `src/pages/index.astro` und `src/pages/en/index.astro` den Produktfilter erweitern:

```
const products = (await getCollection('products', (e) =>
  e.data.lang === lang && e.data.slug !== 'connect')) // ← blendet connect aus
```

Komplett offline nehmen: zusätzlich den Dropdown-Link in `Header.astro` entfernen. (Die Produkt-Markdown-Datei löschen entfernt auch die Seite selbst.)

R5 – Neuen Top-Level-Menüpunkt mit eigener Seite

1. Seiten anlegen: `src/pages/karriere.astro` und `src/pages/en/career.astro` – einfachste Vorlage:

```
---
import SiteLayout from '../layouts/SiteLayout.astro';
---
<SiteLayout title="Karriere – Teraport" lang="de" altHref="/en/career/">
  <article class="tp-container prose">
    <h1>Karriere</h1>
    <p>Inhalt ...</p>
  </article>
</SiteLayout>
```

(EN-Datei: `lang="en"`, `altHref="/karriere/"`, Import mit `../..`)

2. Routenpaar in `routes.ts`: `{ de: '/karriere/', en: '/en/career/' },`
3. Beschriftung in `ui.ts` (beide Sprachen): `'nav.career': 'Karriere' / 'Careers'`

4. Menülink in `Header.astro` an gewünschter Position: `<a href={p('/karriere/', '/en/career/')}>{t(lang, 'nav.career')}</a>`
5. `npm test` && `npm run build` und Vorschau prüfen

R6 – Footer-Links ändern

`src/components/Footer.astro`: Links im `<nav aria-label="Rechtliches">`-Block ergänzen/ändern – gleiche `p('/de-pfad/', '/en-pfad/')`-Systematik wie im Header.

Nach jeder Strukturänderung: `npm test` ausführen – die Tests in `tests/i18n-routes.test.ts` prüfen die Routenpaare. Beim Entfernen einer Route dort auch die zugehörige Testzeile entfernen; beim Hinzufügen empfiehlt sich eine neue Testzeile nach vorhandenem Muster.